

**Universidade Federal de São Carlos (UFSCar)
campus Sorocaba**

Bacharelado em Ciência da Computação

Processamento Massivo de Dados

**Projeto Prático
Fase Final**

Grupo 6
Tema: Curadoria de Livros

Professora responsável: Prof^a. Dr^a. Sahudy Montenegro González

Integrantes do grupo:

André Luís de Souza Oliveira RA 770985

Cinthia Souza Costa RA 792173

03 de Setembro de 2024

ÍNDICE

1. Descrição do tema	3
2. Consultas	6
3. Fonte dos dados	6
4. Fluxograma dos dados	7
5. Transformação dos dados	8
Ingestão dos dados	15
Consultas	19
Desafios e Dificuldades	21
Referências bibliográficas	21

1. Descrição do tema

No Brasil, metade da população tem hábito de ler e boa parte desses leitores brasileiros consomem mais livros com temáticas religiosas [4].

Dados recentes mostram que a leitura continua sendo uma atividade valorizada, especialmente em tempos de maior acesso a informações digitais e a crescente popularidade de livros em formato eletrônico [4]. Mas existem ainda alguns empecilhos que dificultam o crescimento do hábito de leitura do brasileiro vão desde divergências sociais e acesso limitado a livros digitais ou físicos ao pouco estímulo à leitura nas escolas.

Em 2018 apenas 9.5% dos alunos de 15 a 16 anos haviam lido mais de 100 páginas segundo uma pesquisa realizada pelo Centro de Pesquisas em Educação, Interdisciplinaridade e Evidências no Debate Educacional (Iede) e 66% dos estudantes não liam livros com mais de 10 páginas [3].

A presente aplicação tem como objetivo ser um curador literário personalizado, atuando na recomendação personalizada de livros para incentivar e fomentar o hábito de leitura entre os usuários, além de identificar tendências e oportunidades de recomendação de leitura para grupos de usuários.

Uma das principais funcionalidades da aplicação será a recomendação de títulos que será baseada na proximidade entre gêneros literários, autores, similaridade entre livros e avaliações de outros usuários sobre as publicações e também tendências de leitura dentro da comunidade que possa oferecer ao usuário opções diversificadas dentro de seu histórico de leitura.

A partir de avaliações feitas pelos usuários da comunidade sobre suas leituras será possível ainda entender os padrões de consumo literário da comunidade além de tendências de crescimento de público para determinados nichos literários que alimentarão recomendações de livros.

Para isso serão armazenados no banco de dados informações sobre as principais entidades relacionadas no sistema. O Livro é um dos elementos chave da aplicação. Cada livro possui atributos como título, sinopse e a média das avaliações dos

usuários, é relacionado a pelo menos um Gênero literário e escrito por pelo menos um Autor.

O Gênero é a categoria na qual o livro se encaixa podendo o livro ser classificado como fantasia e/ou distopia, biografia entre outros. A Avaliação é a componente que explicita a opinião de um Usuário sobre o Livro lido, essa avaliação é formada por uma nota e pode incluir um comentário.

O Usuário é outro elemento central no qual a aplicação foca, ele lê e interage com os Livros armazenados na plataforma, interações e avaliações essas usadas para alimentar um perfil de preferências literárias que vai ditar as obras recomendadas ao usuário.

Essas entidades formam uma rede de relações que permite a recomendação de títulos com base em preferências e entender hábitos de consumo dos usuários.

O esquema contido na imagem 1 ilustra as relações consideradas como base para o sistema de recomendação.

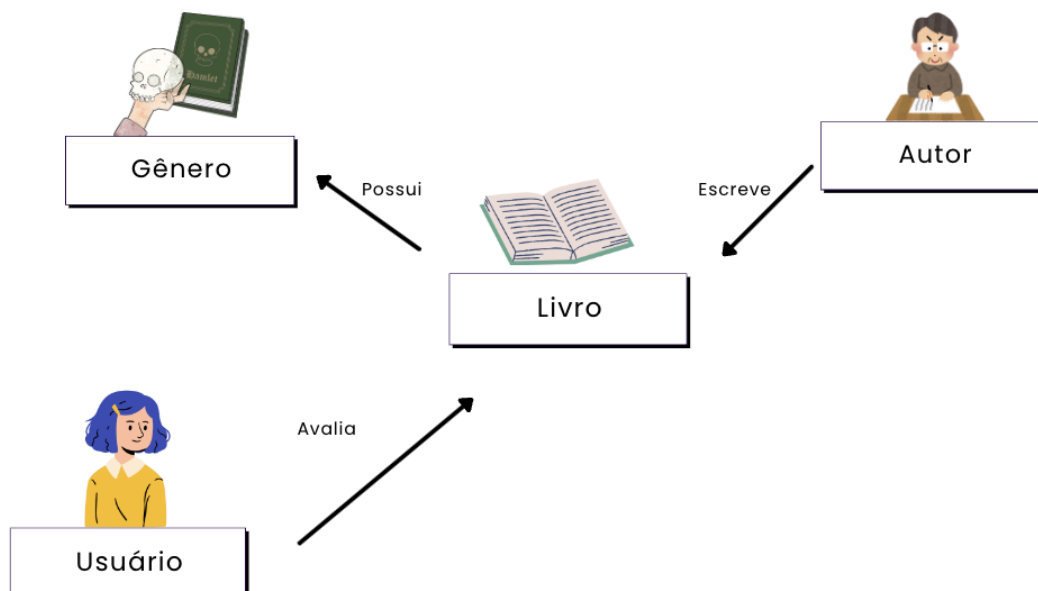
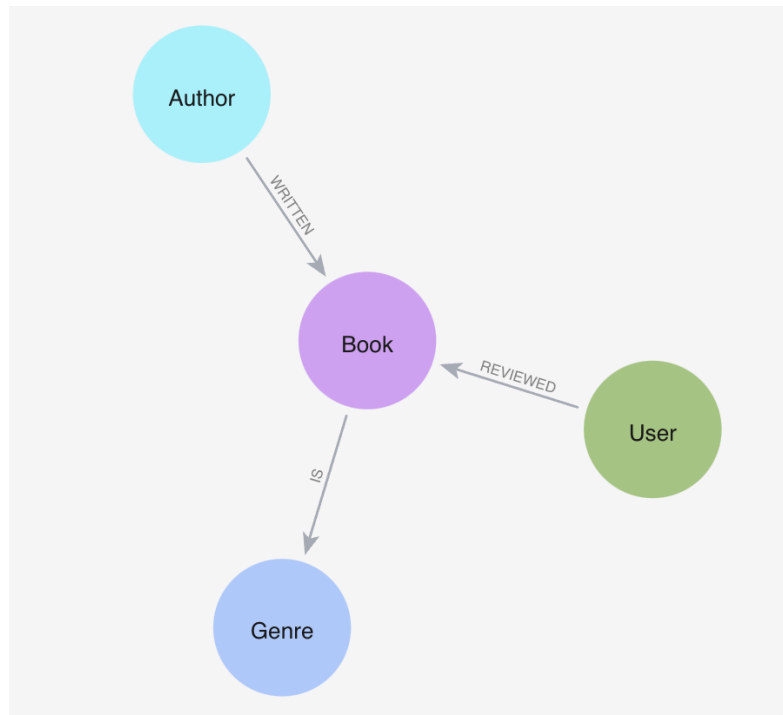


Imagem 1: Esquema dos dados



Esquema de dados pelo AuraDB

2. Consultas

As seguintes consultas foram elencadas para responder a questões pertinentes sobre a aplicação que impactam diretamente na funcionalidade de recomendação.

I. Recomendação com base em usuários semelhantes

Recomendação de títulos com base em livros bem avaliados por usuários que tenham similaridade com o usuário atual.

II. Identificação de tendências

Apresentar tendências de leitura entre usuários ao longo dos anos, identificando flutuações na popularidade de gêneros mais consumidos e bem avaliados em determinados momentos do ano, como feriados, datas comemorativas (23 Abril, Dia Mundial do Livro), entre outros. Buscando identificar possíveis sazonalidades em que alguns gêneros são mais bem avaliados ou menos avaliados.

3. Fonte dos dados

Para alimentar o sistema de recomendação foi selecionado um dataset contendo detalhes e avaliações de livros da Amazon com informações de Maio de 1996 a Julho de 2014. O dataset é dividido em dois arquivos: books_data.csv com detalhes para cada publicação e book_rating.csv, esses arquivos contém cerca de 3 milhões de avaliações e mais de 200 mil títulos. As tabelas 1 e 2 indicam os campos que cada tabela possui e suas respectivas descrições.

Campo	Descrição
id	Id do livro
title	Título do livro
price	Preço
user_id	Id do usuário
profileName	Nome do perfil do usuário
review/helpfulness	avaliação de utilidade da avaliação
review/score	Avaliação de 0 a 5 do livro
review/time	Tempo empregado para a avaliação do livro
review/summary	Resumo do texto de avaliação
review/text	Texto completo da avaliação

Tabela 1: Campos e suas descrições do arquivo books_rating.csv

Campo	Descrição
title	Título do livro
description	Descrição do livro
authors	Nome dos autores do livro
image	Url para a capa do livro
previewLink	Link para acessar o livro no Google Books
publisher	Nome da editora

publishedDate	Data de publicação
infoLink	Link para obter mais informações sobre o livro no google books
categories	Gêneros do livro
ratingsCount	Média das avaliações do livro

Tabela 2: Campos e suas descrições do arquivo book_data.csv

Os dados usados podem ser encontrados na plataforma kaggle no seguinte [link](#) [2] .

4. Fluxograma dos dados

Para o processamento dos dados, utilizaremos o Apache Spark por meio do serviço de cloud Databricks como ferramenta de ETL (Extract, Transform and Load) manipulando os dados fazendo as transformações necessárias para preparar os dados para popular o sistema de recomendações.

O banco de dados escolhido como ferramenta de armazenamento foi Neo4j AuraDB. A escolha foi motivada pela flexibilidade desse banco para representar modelos de relações complexas presentes no sistema e o suporte oferecido para criação de sistemas de recomendação usando a similaridade entre nós [1] e otimização para consultas com níveis profundos de relacionamentos.

O fluxograma abaixo mostra como relacionamos o uso das tecnologias para responder às questões propostas.

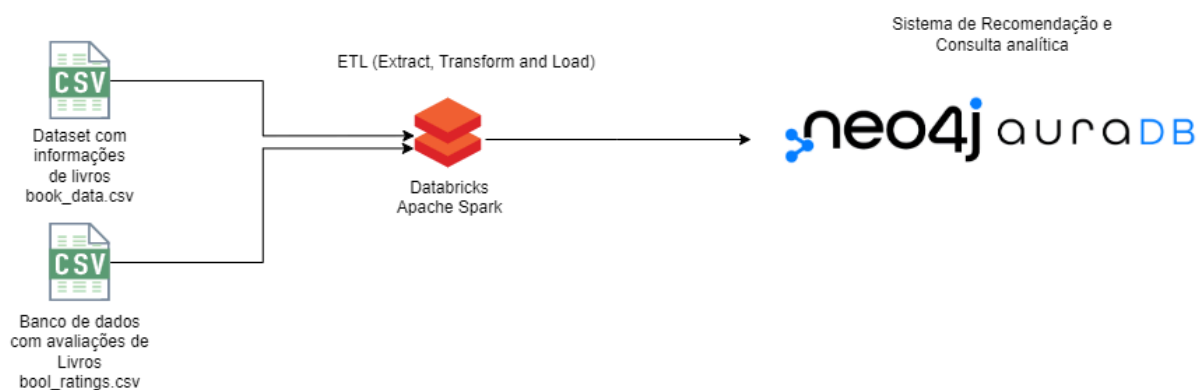


Imagem 1: Fluxo dos dados

5. Transformação dos dados

Para a construção do banco de dados no Neo4j, foram utilizados dos arquivos como base: books_data.csv e books_ratings.csv. Como descrito anteriormente nas tabelas 1 e 2, o primeiro possui informações sobre livros e o segundo informações sobre as avaliações sobre estes livros.

O processo de ETL foi executado usando PySpark no Databricks, por o Databricks ter um limite de upload de arquivos de 2GB o dataset books_ratings.csv foi dividido em 2 arquivos chamados: parte1_Books_rating.csv e parte2_Books_rating.csv. Com 1,33 GB cada.

```
df1 = spark.read.format("csv").option("header", "true").load("dbfs:/FileStore/shared_uploads/pmd/parte2_Books_rating.csv")
df2 = spark.read.format("csv").option("header", "true").load("dbfs:/FileStore/shared_uploads/pmd/parte1_Books_rating.csv")
df3 = spark.read.format("csv").option("header", "true").load("dbfs:/FileStore/shared_uploads/pmd/books_data.csv")
```

Após o carregamento dos arquivos parte1_Books_rating.csv e parte2_Books_rating.csv, eles foram unidos em um *dataframe Spark*.

```
df_reviews = df1.union(df2)
```

Com os dados carregados em dataframes os campos foram analisados previamente para decidir o tipo de tratamento e limpeza que seria feito no feito nos conjuntos de dados.

1) Alterações de data

Foi alterado o tipo e formato da coluna review/time de timestamp para date no formato 'YYYY-MM-DD'. A coluna foi renomeada para review_date.

```
df_reviews = df_reviews.withColumn("date", from_unixtime(col("review/time")).cast("timestamp"))
df_reviews = df_reviews.withColumn("review_date", date_format(col("date"), 'yyyy-MM-dd'))
```

2) Reviews identificados

Algumas das resenhas não apresentavam nenhuma identificação dos autores e por isso foram retiradas, restando apenas os reviews com identificação da autoria.

```
df_reviews = df_reviews.select(
    col("id"),
    col("title"),
    col("price"),
    col("user_id"),
    col("profileName"),
    col("review/score"),
    col("review_date"),
    col("review/summary"),
    col("review/text"),
    col("review/helpfulness")
).filter(col("user_id").isNotNull())
```

3) Utilidade do review

Na coluna review/helpfulness foi feito tratamento para substituir valores inadequados que não indicassem a utilidade da resenha, strings aleatórias foram removidas e substituídas por 0/0 considerando que aquela resenha ainda não havia sido avaliada por outros usuários como útil. Ainda foi estabelecido um valor real para caracterizar o nível de utilidade da resenha variando de 0 a 1, usando como base as informações do próprio campo.

```
# tratamento de strings aleatórias para "0/0"
df_reviews = df_reviews.withColumn("review/helpfulness",
    when(
        col("review/helpfulness").rlike(r"^[^\d]*$"),
        regexp_replace("review/helpfulness", r".*", r"0/0" )
    ).otherwise(col("review/helpfulness")))

# extração dos valores inteiros do campo e transformação em número real
def to_real (helpfulness):
    if helpfulness is None or not isinstance(helpfulness, str):
        return 0.0;

    try:
        numbers = helpfulness.split('/')
        num = float(numbers[0])
        den = float (numbers[1])

        if (den == 0):
            return 0.0

        return num/den
        #return (round(num/den, 2))

    except ValueError:
        return 0.0

to_real_func = udf (to_real, DoubleType())
df_reviews = df_reviews.withColumn("helpfulness", round(to_real_func("review/helpfulness"), 2))
```

4) Conversão de tipo para nota

As notas das avaliações dos livros foram transformadas para o tipo real Double, para facilitar aplicação de funções de cálculo sobre o campo.

```
df_reviews = df_reviews.withColumn("review/score", col("review/score").cast(DoubleType()))
```

5) Excluindo colunas image e previewLink

Para o tratamento do arquivo book_data.csv, foi observado que os links presentes na coluna previewLink e infoLink levavam ao mesmo destino, assim optamos por deletar a coluna previewLink e manter a infoLink. Consequentemente, o site de destino também possui uma imagem de capa do livro, assim excluímos a coluna image.

```
df_book = df3.drop('image', 'previewLink')
```

6) Livro sem título

Percebeu-se que havia um livro sem título, como a coluna Title é importante para saber o livro, foi adicionado o título do livro

```
df_book = df_book.withColumn('Title', when(col('Title').isNull(), 'Nan Yar: Who Am I?').otherwise(col('Title')))
```

7) Conversão do tipo da coluna ratingsCount

Uma função chamada `transforme_string_to_float` é definida e aplicada ao DataFrame. Esta função converte os valores da coluna 'ratingsCount' para o tipo float. Se a conversão resultar em null, o valor é substituído por 0.0.

```
def transforme_string_to_float(df):  
    return df.withColumn(  
        'ratingsCount',  
        when(col('ratingsCount').cast(FloatType()).isNull(), 0.0) # Se o cast para float resultar em nulo, substituir por 0.0  
        .otherwise(col('ratingsCount').cast(FloatType())) # Caso contrário, manter o valor convertido para float  
    )  
  
df_book = transforme_string_to_float(df_book)
```

8) Conversão de strings em array

As colunas 'categories' e 'authors' são verificadas para ver se seguem um padrão específico (iniciando e terminando com '[') que indicasse que o campo fosse um array. Se não corresponderem a esse padrão, são substituídas por uma string padrão '[''] e nenhum gênero ou autor seria associado a obra.

```
df_book = df_book.withColumn('categories', when(col('categories').rlike("^\\[\\[\\'", col('categories')).otherwise("[\\'\\'"])  
df_book = df_book.withColumn('authors', when(col('authors').rlike("^\\[\\[\\'", col('authors')).otherwise("[\\'\\'"])
```

9) Padronização da data de publicação

A coluna 'publishedDate' é verificada para garantir que os valores sejam válidos. Primeiro, ela é limpa para manter apenas datas no formato numérico (ex: YYYY-MM-DD, YYYY-MM e YYYY). Foi utilizado apenas o ano da publicação para garantir maior padronização dos dados.

Se não encaixar nesses padrões é inserido a string “Sem informação”.

```
df_book = df_book.withColumn('publishedDate', when(col('publishedDate').rlike("^\\d{4}(-\\d{2}){0,2}$"), col('publishedDate')).otherwise(None))  
  
df_book = df_book.withColumn("publishedDate",  
    when(length(col("publishedDate")) == 4, col("publishedDate"))  
    .when(col("publishedDate").contains("-"), split(col("publishedDate"), "-").getItem(0)) # Se contém '-', assum  
    .otherwise("sem informacao"))  
  
df_book = df_book.na.fill(["Sem informação"])
```

10) Padronização das categorias e autores

As colunas 'categories' e 'authors' são processadas para remover caracteres indesejados ([,], ') e dividir os valores em listas, usando a vírgula como delimitador. Isso transforma as strings de categorias e autores em listas de valores separados.

```
df_book = df_book.withColumn(
    "categories",
    split(regex_replace(col("categories"), r"[\[\]]", ""), ",\s*")
).withColumn(
    "authors",
    split(regex_replace(col("authors"), r"[\[\]]", ""), ",\s*")
)
```

11) Seleção aleatória de avaliações para escolha de usuários
É selecionada uma amostra aleatória de 75% dos registros do DataFrame de avaliações, limitando a 10.000 registros.

```
df_sample_reviews = df_reviews.sample(fraction = 0.0075).limit(10000)
```

12) Criação do DataFrame de usuários
É extraído a coluna user_id do DataFrame de avaliações para criar o novo DataFrame df_users, que será a base para a criação dos nós de usuários. A biblioteca Faker é usada para gerar dados fictícios para os usuários, nome e endereço. O endereço está no padrão americano. Que foram inseridos no df_users.

```
df_users = df_sample_reviews.select(
    col("user_id")
).distinct()
num_usuarios = df_users.count()
print(num_usuarios)

from faker import Faker
fake = Faker()
# geracao de nomes para os usuarios
def gen_name():
    return fake.name()
def gen_address():
    return fake.address()

gen_name_func = udf(gen_name, StringType())
gen_address_func = udf(gen_address, StringType())

df_users = df_users.withColumn("name", gen_name_func()).withColumn("address", gen_address_func())

display(df_users)
```

13) Seleção aleatória de avaliações
Seleciona uma nova amostra aleatória de 90% dos registros limitando a 8500 registros.

```
df_reviews2 = df_sample_reviews.sample(fraction = 0.9).limit(8500)
```

14) Seleção de livros avaliados

Seleção de livros com títulos distintos.

```
# amostra de livros que foram avaliados
df_books_reviews = df_sample_reviews.select(col("title").distinct())
```

15) Seleção de livros avaliados por usuários

É realizado um inner join entre o DataFrames de livros e o de livros avaliados, assim filtrando os livros que ambos têm em comum. Depois é selecionado as colunas necessárias

```
df_books_validos = df_book.join(df_books_reviews, df_book["title"] == df_books_reviews["title"].alias("nome"), "inner")
df_books_validos = df_books_validos.select([
    df_book["title"],
    col("description"),
    col("authors"),
    col("publishedDate"),
    col("categories"),
    col("infoLink"),
    col("description")
])
```

16) Extração de autores e gêneros dos livros

Expansão das colunas authors e categories usando a função explode, separando as listas nestas colunas em linhas separadas.

É criado o DataFrame df_autores para os autores, selecionando somente os autores com nome que se encaixam no padrão ASCII, isso é feito usando regex.

Também é criado o DataFrame df_genre para os gêneros dos livros, renomeando a coluna categories para genre.

```
df_books_validos = df_books_validos.withColumn("authors", explode(col("authors"))).withColumn("categories", explode(col("categories")))
```

```
# dos livros validos vamos extrair autores e generos

df_autores = df_books_validos.select(
    col("authors").alias("author")
).filter(regex_extract(col("author"), r'^[\x00-\x7F]', 0) == "")

df_genre = df_books_validos.select(
    col("categories").alias("genre")
)
```

17) Seleção de livros distintos após a expansão

Após o uso do explode em autores e categorias, acarretou na duplicação de registros dos livros, sendo necessário a filtragem por livros únicos.

```
df_book_nodes = df_books_validos.select(
    col("title"),
    col("description"),
    col("publishedDate")
).distinct()
```

18) Filtragem de livros com títulos válidos

Observou-se que alguns livros possuíam títulos, descrição, editora e link de informações em alfabeto russo que impedia a inserção de dados no Neo4j. Sendo necessário a execução de regex para filtragem de textos que se encaixem no padrão ASCII.

```
df_books_validos = df_books_validos.filter(((regexp_extract(col("authors"), r'^\x00-\x7F', 0) == "")
& (regexp_extract(col("title"), r'^\x00-\x7F', 0) == "") & (regexp_extract(col("description"), r'^\x00-\x7F', 0) == "")
& (regexp_extract(col("publisher"), r'^\x00-\x7F', 0) == "") & (regexp_extract(col("infoLink"), r'^\x00-\x7F', 0) == ""))).distinct()
```

19) Filtragem de autores e gêneros vazios

Selecionando os registros únicos onde “author” e “genre” não são vazios.

```
df_autores = df_autores.filter(col("author") != "").distinct()
df_genre = df_genre.filter(col("genre") != "").distinct()
```

20) Criação do DataFrame df_evaluation

É criado uma nova coluna "ev_id" que concatena os valores das colunas "id" e "user_id", criando um identificador único para cada avaliação. Selecionando e renomeando as colunas que serão armazenadas.

```
df_evaluation = df_reviews2.withColumn("ev_id", concat(col("id"), col("user_id"))).select(
    col("ev_id"),
    col("user_id"),
    col("title").alias("book_id"),
    col("review_date").alias("date"),
    col("review/score").alias("score"),
    col("helpfulness")
).distinct()
```

É realizado um inner join entre df_evaluation e df_books_validos, com o objetivo de ter apenas as avaliações para as quais existe um título em df_books_validos sejam mantidos. A coluna book_id é renomeada para title.

```
df_evaluation = df_evaluation.join(df_books_validos, df_evaluation["book_id"] == df_books_validos["title"], "inner")
df_evaluation = df_evaluation.select(
    col("book_id").alias("title"),
    col("user_id"),
    col("date"),
    col("score"),
    col("helpfulness"))
```

21) Criação do DataFrame df_written

Selecionando title e authors, que passa a ser author, evitando os autores duplicados

```
df_written = df_books_validos.select(
    col("title"),
    col("authors").alias("author")
).filter(col("author") != "").distinct()
```

22) Criação do DataFrame df_is_genre

Selecionando title e categories, que passa a ser genre, evitando gêneros duplicados

```
df_is_genre = df_books_validos.select([
    col("title"),
    col("categories").alias("genre")
]).filter(col("genre") != "").distinct()
```

Ingestão dos dados

A partir das transformações foram construídos alguns dataframes com campos úteis para nós e relações.

df_genres: data frame contendo os gêneros possíveis encontrados nos livros do data frame df_book. Contém um único campo genre e foi usado para criar os nós de Gênero presentes no banco de dados.

df_authors: se assemelha muito ao df_genre, a única diferença é o campo author usando na criação dos nós do tipo Autor.

df_book_nodes: possui os campos title, description e publishedDate e foi usado para confeccionar os nós de Livros.

df_users: contém o campo de user_id herdado do data frame de reviews e alguns campos adicionais como name e address criados artificialmente a partir da biblioteca Faker. Usado para criação dos nós de usuário.

df_is_genre: continha o title e o genre associado, usado para criar a relação entre gênero e livro

df_written: com campo title e author, usado na relação entre livro e autor

df_evaluation: representando as reviews tem como campos user_id, title, score, helpfulness, date usado para estabelecer a avaliação feita por um usuário sobre um livro.

Esses data frames foram usados como parâmetros para as funções que eram responsáveis pela população do banco de dados. Essas funções criavam uma conexão com o banco de dados Aura BD e executavam queries em linguagem Cypher usando os dados contidos nos campos dos data frames apresentados acima para preencher parâmetros predefinidos dentro das consultas.

I) Create_user_node

```
# função para criar nós de usuários usando o df
def create_user_node (df):

    # criando conexão
    driver = GraphDatabase.driver(uri, auth=(username, password))

    def create_node (tx, user_id, name, address):
        query = (
            "CREATE (u: User {user_id: $user_id, name: $name, address: $address})"
        )
        tx.run(query, user_id = user_id, name = name, address = address)

    with driver.session() as session:
        rows = df.collect()
        for row in rows:

            user_id = row["user_id"]
            name = row["name"]
            address = row["address"]

            session.execute_write(create_node, user_id, name, address)
```

II) Create_book_node

```
def create_book_node (df):

    # criando conexão
    driver = GraphDatabase.driver(uri, auth=(username, password))

    def create_node (tx, title, description, publishedDate):
        query = (
            "CREATE (b: Book {title: $title, description: $description, publishedDate: $publishedDate})"
        )
        tx.run(query, title = title, description = description, publishedDate = publishedDate)

    with driver.session() as session:
        rows = df.collect()
        for row in rows:

            title = row["title"]
            description = row["description"],
            publishedDate = row["publishedDate"],

            session.execute_write(create_node, title, description, publishedDate)
```

III) Create_genre_node

```
# função para criar generos

def create_genre_node (df):

    # criando conexão
    driver = GraphDatabase.driver(uri, auth=(username, password))

    def create_node (tx, genre):
        query = (
            "CREATE (g: Genre {genre: $genre})"
        )
        tx.run(query, genre = genre)

    with driver.session() as session:
        rows = df.collect()
        for row in rows:

            genre = row["genre"]

            session.execute_write(create_node, genre)
```

IV) Create_author_node

```
# função para criar autores

def create_author_node (df):

    # criando conexão
    driver = GraphDatabase.driver(uri, auth=(username, password))

    def create_node (tx, author):
        query = (
            "CREATE (a: Author {author: $author})"
        )
        tx.run(query, author = author)

    with driver.session() as session:
        rows = df.collect()
        for row in rows:

            author = row["author"]

            session.execute_write(create_node, author)
```

V) Create_is_genre_relationship

```
# função para criar relações de "tem gênero" (livro - [tem] -> genero) escrever

def create_genre_relationship (df):
    # criando conexão
    driver = GraphDatabase.driver(uri, auth=(username, password))

    def create_relationship (tx, title, genre):
        query = (
            "MATCH (b:Book {title: $title}), (g:Genre {genre: $genre}) "
            "MERGE (b)-[r:IS]->(g) "
            "RETURN b, r, g"
        )
        tx.run(query, title = row["title"], genre = row["genre"])

    with driver.session() as session:
        rows = df.collect()
        for row in rows:

            title = row["title"]
            genre = row["genre"]

            session.execute_write(create_relationship, title, genre)
```

VI) Create_written_relationship

```
# função para criar relações de escrita (autor - [escreveu] -> livro)

def create_written_relationship (df):
    # criando conexão
    driver = GraphDatabase.driver(uri, auth=(username, password))

    def create_relationship (tx, author, title):
        query = (
            "MATCH (a:Author {author: $author}), (b:Book {title: $title}) "
            "MERGE (a)-[r:WRITTEN]->(b) "
            "RETURN a, r, b"
        )
        tx.run(query, author = row["author"], title = row["title"])

    with driver.session() as session:
        rows = df.collect()
        for row in rows:

            author = row["author"]
            title = row["title"]

            session.execute_write(create_relationship, author, title)
```

VII) Create_reviewed_relationship

```
# função para criar relações de avaliações ( usuário - [avaliou] -> livro )
def create_review_relationship(df):
    # criando conexão
    driver = GraphDatabase.driver(uri, auth=(username, password))

    def create_relationship(tx, date, user_id, title, score, helpfulness):
        query = (
            "MATCH (u:User {user_id: $user_id}), (b:Book {title: $title}) "
            "MERGE (u)-[r:REVIEWED]->(b) "
            "SET r.date = $date, r.title = $title, r.score= $score, r.helpfulness = $helpfulness "
            "RETURN u, r, b"
        )
        tx.run(query, date = row["date"], user_id = row["user_id"], title = row["title"], score = row["score"],
            helpfulness = row["helpfulness"])

    with driver.session() as session:
        rows = df.collect()
        for row in rows:

            date = row["date"]
            user_id = row["user_id"]
            title = row["title"]
            score = row["score"]
            helpfulness = row["helpfulness"]

            session.execute_write(create_relationship, date, user_id, title, score, helpfulness)
```

Consultas

1) Recomendação de livros para usuários semelhantes

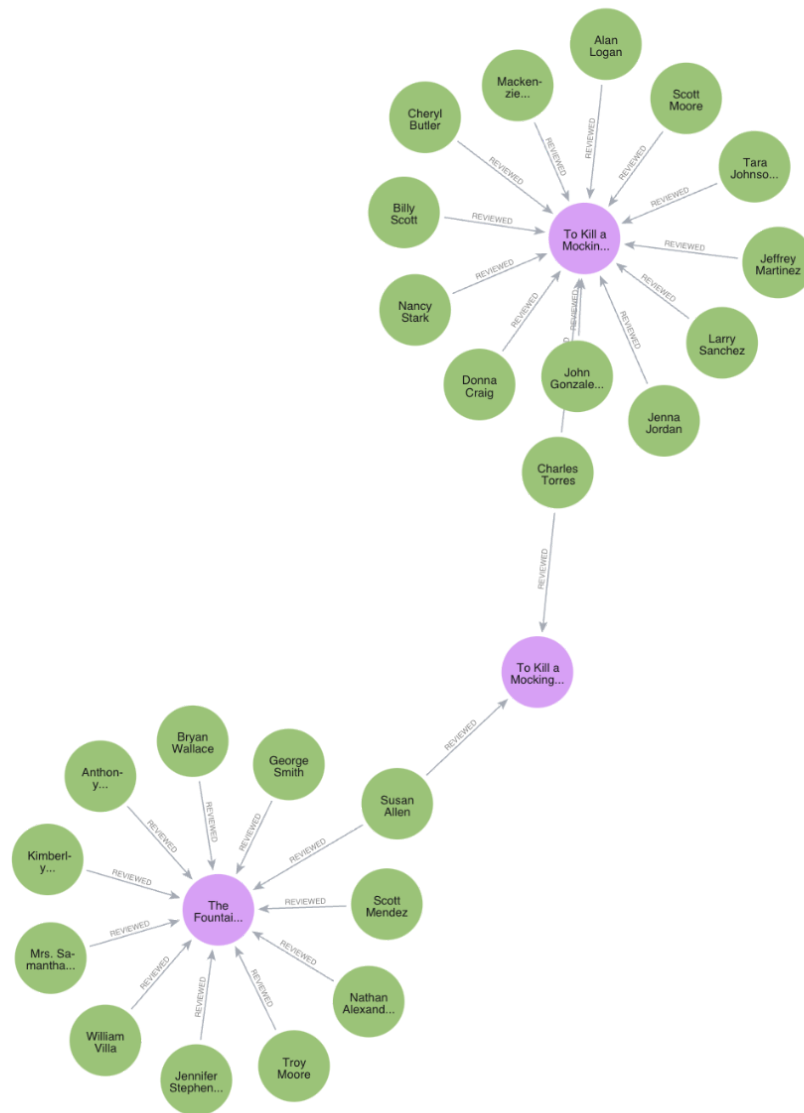
Foi utilizado como critério de similaridade a diferença entre a pontuação dada a um mesmo livro por usuários diferentes, quanto menor a diferença de pontuação mais os usuários terão um perfil parecido.

```
1 MATCH (u1:User)-[r1: REVIEWED]->(b:Book)->[r2: REVIEWED]-(u2:User)
2 WHERE u1 <> u2
3 WITH u1, u2, AVG(ABS(r1.score - r2.score)) AS delta, COUNT(b) AS livros
4
5 MATCH (u1) - [r3: REVIEWED] -> (b1: Book)
6 WHERE NOT (u2) - [: REVIEWED] -> (b1: Book)
7 AND r3.score > 3
8 AND r3.helpfulness > 0
9 AND delta < 1 // criterio similaridade
10 RETURN u2.name AS Usuario, b1.title AS Recomendação, delta
11 ORDER BY delta ASC, livros DESC
12
```

Resultado tabular da query:

	Usuario	Recomendação	delta	livros
1	"Troy Moore"	"To Kill a Mockingbird"	0.0	1
2	"Nathan Alexander"	"To Kill a Mockingbird"	0.0	1
3	"George Smith"	"To Kill a Mockingbird"	0.0	1
4	"Bryan Wallace"	"To Kill a Mockingbird"	0.0	1
5	"Anthony Edwards"	"To Kill a Mockingbird"	0.0	1
6	"Kimberly Romero"	"To Kill a Mockingbird"	0.0	1
7	"William Villa"	"To Kill a Mockingbird"	0.0	1
8	"Sheri Martinez"	"Of Mice & Men"	0.0	1
9	"Alvin Riley"	"Of Mice & Men"	0.0	1

Na imagem abaixo conseguimos enxergar visualmente como "To Kill a Mockingbird" seria recomendado aos leitores de "The Fountainhead".



2) Identificação de tendência

A consulta responde a pergunta: Qual o gênero mais bem avaliado em cada ano? Essa consulta encontra o gênero mais popular para cada ano, baseado nas revisões bem avaliadas, e retorna a quantidade de livros e a média do score para cada gênero mais popular em cada ano.

```

1 MATCH (u:User)-[r:REVIEWED]->(b:Book)-[:IS]->(g:Genre)
2 WHERE r.score >= 4
3 WITH g.genre as genre, substring(toString(r.date), 0, 4) AS year, avg(r.score) AS avgScore, count(*) AS genreCount
4 ORDER BY year, genreCount DESC
5
6 WITH year, collect({genre: genre, count: genreCount, avgScore: avgScore}) AS genres
7 RETURN year as Ano, genres[0].genre AS GeneroMaisPopular, genres[0].count AS Qtd, genres[0].avgScore AS MediaScore
8 ORDER BY year;

```

	Ano ≡	GeneroMaisPopular	Qtd	MediaScore
1	"1997"	"Fiction"	2	5.0
2	"1998"	"Fiction"	18	4.722222222222222
3	"1999"	"Fiction"	34	4.823529411764705
4	"2000"	"Fiction"	62	4.70967741935484
5	"2001"	"Fiction"	70	4.642857142857144
6	"2002"	"Fiction"	54	4.703703703703705
7	"2003"	"Fiction"	50	4.680000000000003
8	"2004"	"Fiction"	76	4.671052631578947
9	"2005"	"Fiction" 	53	4.811320754716981
10	"2006"	"Fiction"	59	4.762711864406779
11	"2007"	"Fiction"	49	4.673469387755101
12	"2008"	"Fiction"	41	4.560975609756097
13	"2009"	"Fiction"	38	4.763157894736843
14	"2010"	"Fiction"	36	4.750000000000001
15	"2011"	"Fiction"	59	4.677966101694914
16	"2012"	"Fiction"	115	4.721739130434783
17	"2013"	"Fiction"	90	4.722222222222225

Desafios e Dificuldades

Pelo fato de nossos dados serem grandes, tivemos que particionar um dos nossos datasets para fazer o upload no databricks. Em alguns campos nos dois datasets tinham dados faltantes ou informações que não coincidiam com os valores que deveriam estar na coluna, levando um tempo para identificar e preencher esses campos com outros valores. Por o AuraDB ter um limite de 200 mil nós e 400 mil relacionamentos, tivemos que diminuir a quantidade de dados inseridos, esse processo das inserções no banco de dados levou bastante tempo para serem concluídas.

Criar consultas Cypher para um sistema de recomendação de livros baseado em filtragem colaborativa e análise das tendências de avaliações dos livros mais bem avaliados por ano apresentou uma série de desafios. A complexidade da linguagem Cypher, combinada com a estrutura dos dados e as consultas avançadas necessárias, tornou o processo desafiador. A dificuldade surgiu principalmente ao tentar extrair e manipular informações específicas, como a média das avaliações e a identificação dos gêneros mais populares ao longo dos anos. Apesar desses obstáculos, a elaboração das consultas permitiu uma visão detalhada e valiosa das preferências e tendências dos leitores.

Referências bibliográficas

- [1] Neo4j Graph Data Science: Disponível em [Neo4j Graph Data Science](#).
- [2] Amazon books reviews: Disponível em [Kaggle](#).
- [3] 66% dos alunos brasileiros não leem textos com mais de dez páginas, diz estudo: Disponível em [CNN Brasil](#).
- [4] Os hábitos de leitura do brasileiro: Disponível em [Globo Gente](#).
- [5] Neo4j Connector for Apache Spark: Disponível em [Neo4j Spark](#).